

Sec. 6.4. John the Ripper

John the Ripper

Task 1 - Introduction

John the Ripper is a well-known, well-loved, and versatile hash-cracking tool. It combines a fast cracking speed with an extraordinary range of compatible hash types.

John the Ripper is an open-source password-cracking tool used to audit password security by attempting to recover plaintext passwords from password hashes. It supports many hash formats and can perform dictionary attacks, brute-force attacks, and other password-cracking techniques.

Task 2 - Basic Terms

A hash is a way of taking a piece of data of any length and representing it in another fixed-length form. This process masks the original value of the data. The hash value is obtained by running the original data through a hashing algorithm. Many popular hashing algorithms exist, such as MD4, MD5, SHA1 and NTLM.

If we take "polo", a string of four characters, and run it through an MD5 hashing algorithm, we end up with an output of `b53759f3ce692de7aff1b5779d3964da`, a standard 32-character MD5 hash.

Likewise, if we take "polomints", a string of 9 characters, and run it through the same MD5 hashing algorithm, we end up with an output of `584b6e4f4586e136bc280f27f9c64f3b`, another standard 32-character MD5 hash.

What Makes Hashes Secure? Hashing functions are designed as one-way functions. In other words, it is easy to calculate the hash value of a given input; however, it is a hard problem to find the original input given the hash value. In simple terms, a hard problem quickly becomes computationally infeasible in computer science. This computational problem has its roots in mathematics as P vs NP. In computer science, P and NP are two classes of problems that help us understand the efficiency of algorithms:

- **P (Polynomial Time):** Class P covers the problems whose solution can be found in polynomial time. Consider sorting a list in increasing order. The longer the list, the longer it would take to sort; however, the increase in time is not exponential.
- **NP (Non-deterministic Polynomial Time):** Problems in the class NP are those for which a given solution can be checked quickly, even though finding the solution itself might be hard. In fact, we don't know if there is a fast algorithm to find the solution in the first place.

Answer the questions below

| What is the most popular extended version of John the Ripper? Jumbo John

Task 3 - Setting Up Your System

Throughout the tasks of this room, we will be using the following :

- The "Jumbo John" version of John the Ripper
- The RockYou password list

If you use the attached lab machine or the AttackBox, you don't need to install John the Ripper on your system. Consequently, feel free to skip through the installation section. If you prefer to use your system to follow along, please read along to learn how to proceed with the installation. We should note that if you use a version of John the Ripper other than Jumbo John, you might not have some of the required tools, such as `zip2john` and `rar2john`.

John the Ripper is supported on many Operating Systems, not just Linux Distributions. Before we go through this, there are multiple versions of John, the standard "core" distribution, and multiple community editions, which extend the feature set of the original John distribution. The most popular of these distributions is the "**Jumbo John**," which we will use specific features of later.

You can double-check this by typing `john` into the terminal. You should be met with a usage guide for John, with the first line reading "John the Ripper 1.9.0-jumbo-1" or something similar with a different version number.

Many Linux distributions have John the Ripper available for installation from their official repositories. For instance, on Fedora Linux, you can install John the Ripper with `sudo dnf install john`, while on Ubuntu, you can install it with `sudo apt install john`. Unfortunately, at the time of writing, these versions provided core functionality and missed some of the tools available through Jumbo John.

Wordlists: Now that we have `john` ready, we must consider another indispensable component: wordlists. On the AttackBox and Kali Linux distributions, the `/usr/share/wordlists` directory contains a series of great wordlists.

RockYou: For all of the tasks in this room, we will use the infamous `rockyou.txt` wordlist, a very large common password wordlist obtained from a data breach on a website called rockyou.com in 2009. If you are not using any of the above distributions, you can get the `rockyou.txt` wordlist from the [SecLists\(opens in new tab\)](#) repository under the `/Passwords/Leaked-Databases` subsection. You may need to extract it from the `.tar.gz` format using `tar xvzf rockyou.txt.tar.gz`.

Answer the questions below

Which website's breach was the rockyou.txt wordlist created from? rockyou.com

Task 4 - Cracking Basic Hashes

There are multiple ways to use John the Ripper to crack simple hashes.

John Basic Syntax: The basic syntax of John the Ripper commands is as follows. We will cover the specific options and modifiers used as we use them.

```
john [options] [file path]
```

- `john`: Invokes the John the Ripper program
- `[options]`: Specifies the options you want to use
- `[file path]`: The file containing the hash you're trying to crack; if it's in the same directory, you won't need to name a path, just the file.

Automatic Cracking: John has built-in features to detect what type of hash it's being given and to select appropriate rules and formats to crack it for you; this isn't always the best idea as it can be unreliable, but if you can't identify what hash type you're working with and want to try cracking it, it can be a good option! To do this, we use the following syntax:

```
john --wordlist=[path to wordlist] [path to file]
```

- `--wordlist=`: Specifies using wordlist mode, reading from the file that you supply in the provided path
- `[path to wordlist]`: The path to the wordlist you're using, as described in the previous task

Example Usage: `john --wordlist=/usr/share/wordlists/rockyou.txt hash_to_crack.txt`

Identifying Hashes: Sometimes, John won't play nicely with automatically recognising and loading hashes, but that's okay! We can use other tools to identify the hash and then set John to a specific format. To use hash-identifier, you can use `wget` or `curl` to download the Python file `hash-id.py` from its GitLab [page\(opens in new tab\)](#). Then, launch it with `python3 hash-id.py` and enter the hash you're trying to identify. It will give you a list of the most probable formats. These two steps are shown in the terminal below.

```
john --format=[format] --wordlist=[path to wordlist] [path to file]
```

- `-format=` : This is the flag to tell John that you're giving it a hash of a specific format and to use the following format to crack it
- `[format]` : The format that the hash is in

A Note on Formats: When you tell John to use formats, if you're dealing with a standard hash type, e.g. md5 as in the example above, you have to prefix it with `raw-` to tell John you're just dealing with a standard hash type, though this doesn't always apply. To check if you need to add the prefix or not, you can list all of John's formats using `john --list=formats` and either check manually or grep for your hash type using something like `john --list=formats | grep -iF "md5"`.

Answer the questions below

```
root@ip-10-113-98-242:~/Rooms/JohnTheRipper# cd Task4
bash: cd: Task4: No such file or directory
root@ip-10-113-98-242:~/Rooms/JohnTheRipper# cd task4
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4# ls
first_task_hashes_1605054698917.zip
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4# unzip first_task_hashes_1605054698917.zip
Archive:  first_task_hashes_1605054698917.zip
  creating: first_task_hashes/
  extracting: first_task_hashes/hash1.txt
  inflating: first_task_hashes/hash3.txt
  extracting: first_task_hashes/hash2.txt
  inflating: first_task_hashes/hash4.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4# ls
first_task_hashes  first_task_hashes_1605054698917.zip
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4# cd first_task_hashes
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4/first_task_hashes# ls
hash1.txt hash2.txt hash3.txt hash4.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4/first_task_hashes# cat has
h1.txt
2e728dd31fb5949bc39cac5a9f066498
```

```
root@lp-10-113-98-242:~/Rooms/JohnTheRipper/task4/flrst_task_hashes# python3 hash-id.py  
#####  
#  
#                               #  
#                               #  
#                               #  
#                               #  
#                               #  
#                               #  
#                               # v1.2  
# By Zion3R #  
# www.Blackexploit.com #  
# Root@Blackpoit.com #  
#####  
  
-----  
HASH: 2e728dd31fb5949bc39cac5a9f066498  
  
Possible Hashes:  
[+] MD5  
[+] Domain Cached Credentials - MD4(MD4(($pass)).(strtolower($username)))
```

```

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4/first_task_hashes# john --
format=raw-md5 --wordlist=/usr/share/wordlists/rockyou.txt hash1.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
biscuit (?)
1g 0:00:00:00 DONE (2026-06-24 07:41) 6.667g/s 17920p/s 17920c/s 17920C/s sk
yblue..nugget
Use the "--show --format=Raw-MD5" options to display all of the cracked pass
words reliably
Session completed

```

What type of hash is hash2.txt? sha1

```

root@ip-10-113-98-242:~/Rooms/JohnTheRipp... x root@ip-10-113-98-242:~/Rooms/JohnTheRippe
HASH: 1A732667F3917C0F4AA98BB13011B9090C6F8065

Possible Hashs:
+] SHA-1
+] MySQL5 - SHA-1(SHA-1($pass))

Least Possible Hashs:
+] Tiger-160
+] Haval-160

```

What is the cracked value of hash2.txt? kangeroo

```

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4/first_task_hashes# john --
wordlist=/usr/share/wordlists/rockyou.txt hash2.txt
Warning: detected hash type "Raw-SHA1", but the string is also recognized as
"Raw-SHA1-AxCrypt"
Use the "--format=Raw-SHA1-AxCrypt" option to force loading these as that ty
pe instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as
"Raw-SHA1-Linkedin"
Use the "--format=Raw-SHA1-Linkedin" option to force loading these as that t
ype instead
Warning: detected hash type "Raw-SHA1", but the string is also recognized as
"ripemd-160"
Use the "--format=ripemd-160" option to force loading these as that type ins
tead
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA1 [SHA1 256/256 AVX2 8x])
Warning: no OpenMP support for this hash type, consider --fork=2
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
kangeroo (?)
1g 0:00:00:00 DONE (2026-06-24 07:45) 5.000g/s 586240p/s 586240c/s 586240C/s
kathy18..jirafita

```

What type of hash is hash3.txt? sha256

```

HASH: D7F4D3CCEE7ACD3DD7FAD3AC2BE2AAE9C44F4E9B7FB802D73136D4C53920140A

Possible Hashs:
[+] SHA-256
[+] Haval-256

Least Possible Hashs:
[+] GOST R 34.11-94
[+] RipeMD-256
[+] SNEFRU-256
[+] SHA-256(HMAC)
[+] Haval-256(HMAC)

```

What is the cracked value of hash3.txt? microphone

```

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4/first_task_hashes# john --
format=raw-sha256 --wordlist=/usr/share/wordlists/rockyou.txt hash3.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-SHA256 [SHA256 256/256 AVX2 8x])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Will run 2 OpenMP threads
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
microphone (?)
1g 0:00:00:00 DONE (2026-06-24 07:52) 5.556g/s 546133p/s 546133c/s 546133C/s
rozalia..Dominic1
Use the "--show --format=Raw-SHA256" options to display all of the cracked p
asswords reliably
Session completed

```

What type of hash is hash4.txt? whirlpool

```

HASH: c5a60cc6bbba781c601c5402755ae1044bbf45b78d1183cbf2ca1c865b6c792cf3c6b
87791344986c8a832a0f9ca8d0b4afd3d9421a149d57075e1b4e93f90bf

Possible Hashs:
[+] SHA-512
[+] Whirlpool

Least Possible Hashs:
[+] SHA-512(HMAC)
[+] Whirlpool(HMAC)

```

What is the cracked value of hash4.txt? colossal

```

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task4/first_task_hashes# john --
format=whirlpool --wordlist=/usr/share/wordlists/rockyou.txt hash4.txt
Using default input encoding: UTF-8
Loaded 1 password hash (whirlpool [WHIRLPOOL 32/64])
Warning: poor OpenMP scalability for this hash type, consider --fork=2
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
colossal (?)
1g 0:00:00:00 DONE (2026-06-24 08:00) 1.587g/s 1079Kp/s 1079Kc/s 1079KC/s co
ldog12..chata1994
Use the "--show" option to display all of the cracked passwords reliably
Session completed

```

Task 5 - Cracking Windows Authentication Hashes

Authentication hashes are the hashed versions of passwords stored by operating systems; it is sometimes possible to crack them using our brute-force methods. To get your hands on these hashes, you must often already be a privileged user, so we will explain some of the hashes we plan on cracking as we attempt them.

NTHash / NTLM: NTHash is the hash format modern Windows operating system machines use to store user and service passwords. It's also commonly referred to as NTLM, which references the previous version of Windows format for hashing passwords known as LM, thus NT/LM.

Practical: Now that you know the theory behind it, see if you can use the techniques we practised in the last task and the knowledge of what type of hash this is to crack the `ntlm.txt` file! The file is located in `~/John-the-Ripper-The-Basics/Task05/`.

Answer the questions below

What do we need to set the `--format` flag to in order to crack this hash? nt

What is the cracked value of this password? mushroom

```

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task5# john --format=nt --w
ordlist=/usr/share/wordlists/rockyou.txt ntlm_1605054722641.txt
Using default input encoding: UTF-8
Loaded 1 password hash (NT [MD4 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Note: Passwords longer than 27 rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for st
atus
mushroom (?)
1g 0:00:00:00 DONE (2026-06-24 08:05) 6.667g/s 20480p/s 20480c/s 20480C
/s skater1..dangerous
Use the "--show --format=NT" options to display all of the cracked pass
words reliably
Session completed

```

Task 6 - Cracking /etc/shadow Hashes

Cracking Hashes from /etc/shadow : The `/etc/shadow` file is the file on Linux machines where password hashes are stored. It also stores other information, such as the date of last password change and password expiration information. It contains one entry per line for each user or user account of the system. This file is usually only accessible by the root user, so you must have sufficient privileges to access the hashes. However, if you do, there is a chance that you will be able to crack some of the hashes.

Unshadowing: John can be very particular about the formats it needs data in to be able to work with it; for this reason, to crack `/etc/shadow` passwords, you must combine it with the `/etc/passwd` file for John to understand the data it's being given. To do this, we use a tool built into the John suite of tools called `unshadow`. The basic syntax of `unshadow` is as follows:

```
unshadow [path to passwd] [path to shadow]
```

- `unshadow` : Invokes the unshadow tool
- `[path to passwd]` : The file that contains the copy of the `/etc/passwd` file you've taken from the lab machine
- `[path to shadow]` : The file that contains the copy of the `/etc/shadow` file you've taken from the lab machine

Example Usage: : `unshadow local_passwd local_shadow > unshadowed.txt`

When using `unshadow`, you can either use the entire `/etc/passwd` and `/etc/shadow` files, assuming you have them available, or you can use the relevant line from each, for example:

FILE 1 - local_passwd : Contains the `/etc/passwd` line for the root user: `root:x:0:0::/root:/bin/bash`

FILE 2 - local_shadow : Contains the `/etc/shadow` line for the root

user: `root:$6$2nwjN454g.dv4HN/$m9Z/r2xVfweYVkr.r.v5Ft8Ws3/YYksfNwq96UL1FX00JjY1L6L.DS3KEVsZ9r0VLB/ldTeEL/0IhJZ4GMFMA0:18576:::`

Cracking: We can then feed the output from `unshadow`, in our example use case called `unshadowed.txt`, directly into John. We should not need to specify a mode here as we have made the input specifically for John; however, in some cases, you will need to specify the format as we have done previously using: `--format=sha512crypt`

```
john --wordlist=/usr/share/wordlists/rockyou.txt --format=sha512crypt unshadowed.txt
```

Practical: Now, see if you can follow the process to crack the password hash of the root user provided in the `etchashes.txt` file. Good luck! The files are located in `~/John-the-Ripper-The-Basics/Task06/`.

Answer the questions below

What is the root password? 1234


```

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# ls
etc_hashes_1605054759028.txt  unshadowed.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# cat unshadowed.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# cat etc_hashes_1605054759028.txt
This is everything I managed to recover from the target machine before
my computer crashed... See if you can crack the hash so we can at least
salvage a password to try and get back in.

root:x:0:0::/root:/bin/bash
root:$6$Ha.d5nGupBm29pYr$yugXSk24ZljLTAZZagtGwpSQhb3F2D0JtnHrvk7HI2ma4G
suioHp8sm3LJiRjPkfIf7LZQ29qgtH17Q/JDpYM/:18576:::::

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# vim local_passwd
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# vim local_shadow
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# unshadow local_passwd
d local_shadow > unshadowed.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# ls
etc_hashes_1605054759028.txt  local_shadow
local_passwd                  unshadowed.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# john --wordlist=/usr
/share/wordlists/rockyou.txt --format=sha512crypt unshadowed.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2
4x])
Cost 1 (iteration count) is 5000 for all loaded hashes
Will run 2 OpenMP threads
Note: Passwords longer than 26 [worst case UTF-8] to 79 [ASCII] rejecte
d
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for st
atus

( )
1g 0:00:00:02 DONE (2026-06-24 08:27) 0.3831g/s 490.4p/s 490.4c/s 490.4
C/s kucing..poohbear1
Use the "--show" option to display all of the cracked passwords reliabl

```

```

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# john --wordlist=/us
/share/wordlists/rockyou.txt --format=sha512crypt unshadowed.txt
Using default input encoding: UTF-8
Loaded 1 password hash (sha512crypt, crypt(3) $6$ [SHA512 256/256 AVX2
4x])
Cracked 1 password hash (is in /usr/local/john/run/john.pot), use "--s
ow"
No password hashes left to crack (see FAQ)
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# john --format=sha51
crypt unshadowed.txt --show
root:1234:0:0::/root:/bin/bash

1 password hash cracked, 0 left

```

Task 7 - Single Crack Mode

So far, we've been using John's wordlist mode to brute-force simple and not-so-simple hashes. But John also has another mode, called the **Single Crack** mode. In this mode, John uses only the information provided in the username to try and work out possible passwords heuristically by slightly changing the letters and numbers contained within the username.

Using Single Crack Mode: To use single crack mode, we use roughly the same syntax that we've used so far; for example, if we wanted to crack the password of the user named "Mike", using the single mode, we'd use: `john --`

```
single --format=[format] [path to file]
```

- `-single` : This flag lets John know you want to use the single hash-cracking mode
- `-format=[format]` : As always, it is vital to identify the proper format.

Example Usage: `john --single --format=raw-sha256 hashes.txt`

A Note on File Formats in Single Crack Mode: If you're cracking hashes in single crack mode, you need to change the file format that you're feeding John for it to understand what data to create a wordlist from. You do this by prepending the hash with the username that the hash belongs to, so according to the above example, we would change the file `hashes.txt`

From `1efee03cdcb96d90ad48ccc7b8666033`

To `mike:1efee03cdcb96d90ad48ccc7b8666033`

Practical: Now that you're familiar with the Syntax for John's single crack mode, access the hash and crack it, assuming that the user it belongs to is called "Joker". The file is located in `~/John-the-Ripper-The-Basics/Task07/`.

Answer the questions below

What is Joker's password? Jok3r

```
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task6# cd
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task7# cd
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task7# ls
hash7_1605054793656.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task7# cat hash7_1605054793656.txt
7bf6d9bb82bed1302f331fc6b816aada
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task7# vim hash7_1605054793656.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task7# vim hash7_1605054793656.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task7# cat hash7_1605054793656.txt
joker:7bf6d9bb82bed1302f331fc6b816aada

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task7# john --single --format=raw-md5 hash7_1605054793656.txt
Using default input encoding: UTF-8
Loaded 1 password hash (Raw-MD5 [MD5 256/256 AVX2 8x3])
Warning: no OpenMP support for this hash type, consider --fork=2
Note: Passwords longer than 18 [worst case UTF-8] to 55 [ASCII] rejected
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
Warning: Only 18 candidates buffered for the current salt, minimum 24 recommended for performance.
Jok3r (joker)
1g 0:00:00:00 DONE (2026-06-24 08:38) 5.882g/s 1152p/s 1152c/s 1152C/s
JOKER5..J0ker
```

Task 8 - Custom Rules

What are Custom Rules? As we explored what John can do in Single Crack Mode, you may have some ideas about some good mangling patterns or what patterns your passwords often use that could be replicated with a particular mangling pattern. The good news is that you can define your rules, which John will use to create passwords dynamically. The ability to define such rules is beneficial when you know more information about the password structure of whatever your target is.

Common Custom Rules Many organisations will require a certain level of password complexity to try and combat dictionary attacks. In other words, when creating a new account or changing your password, if you attempt a password like `polopassword`, it will most likely not work. The reason would be the enforced password complexity. As a result, you may receive a prompt telling you that passwords have to contain at least one character from each of the following:

- Lowercase letter
- Uppercase letter
- Number
- Symbol

Password complexity is good! However, we can exploit the fact that most users will be predictable in the location of these symbols. For the above criteria, many users will use something like the following:

```
Polopassword1!
```

Consider the password with a capital letter first and a number followed by a symbol at the end. This familiar pattern of the password, appended and prepended by modifiers (such as capital letters or symbols), is a memorable pattern that people use and reuse when creating passwords. This pattern can let us exploit password complexity predictability.

Now, this does meet the password complexity requirements; however, as attackers, we can exploit the fact that we know the likely position of these added elements to create dynamic passwords from our wordlists.

How to create Custom Rules: Custom rules are defined in the `john.conf` file. This file can be found in `/opt/john/john.conf` on the TryHackMe Attackbox. It is usually located in `/etc/john/john.conf` if you have installed John using a package manager or built from source with `make`.

The first line: `[List.Rules:THMRules]` is used to define the name of your rule; this is what you will use to call your custom rule a John argument.

We then use a regex style pattern match to define where the word will be modified; again, we will only cover the primary and most common modifiers here:

- `Az` : Takes the word and appends it with the characters you define
- `A0` : Takes the word and prepends it with the characters you define
- `c` : Capitalises the character positionally

These can be used in combination to define where and what in the word you want to modify. Lastly, we must define what characters should be appended, prepended or otherwise included. We do this by adding character sets in square brackets `[ ]` where they should be used. These follow the modifier patterns inside double quotes `"`. Here are some common examples:

- `[0-9]` : Will include numbers 0-9
- `[0]` : Will include only the number 0
- `[A-z]` : Will include both upper and lowercase
- `[A-Z]` : Will include only uppercase letters
- `[a-z]` : Will include only lowercase letters

Please note that:

- `[a]` : Will include only `a`
- `[!f$%@]` : Will include the symbols `!`, `f`, `$`, `%`, and `@`

Putting this all together, to generate a wordlist from the rules that would match the example password `Polopassword1!` (assuming the word `polopassword` was in our wordlist), we would create a rule entry that looks like this:

```
[List.Rules:PoloPassword]
cAz"[0-9] [!f$%@]"
```

Utilises the following:

- `c` : Capitalises the first letter
- `Az` : Appends to the end of the word
- `[0-9]` : A number in the range 0-9
- `[!f$%@]` : The password is followed by one of these symbols

Using Custom Rules:

We could then call this custom rule a John argument using the `--rule=PoloPassword` flag.

As a full command: `john --wordlist=[path to wordlist] --rule=PoloPassword [path to file]`

As a note, I find it helpful to talk out the patterns if you're writing a rule; as shown above, the same applies to writing RegEx patterns. Jumbo John already has an extensive list of custom rules containing modifiers for use in almost all cases. If you get stuck, try looking at those rules [around line 678] if your syntax isn't working correctly.

Answer the questions below

- | What do custom rules allow us to exploit? Password complexity predictability
- | What rule would we use to add all capital letters to the end of the word? `Az"[A-Z]"`
- | What flag would we use to call a custom rule called THMRules? `-rule=THMRules`

Task 9 - Cracking Password Protected Zip Files

We can use John to crack the password on password-protected Zip files. Again, we'll use a separate part of the John suite of tools to convert the Zip file into a format that John will understand, but we'll use the syntax you're already familiar with for all intents and purposes.

Zip2John: Similarly to the `unshadow` tool we used previously, we will use the `zip2john` tool to convert the Zip file into a hash format that John can understand and hopefully crack. The primary usage is like this:

```
zip2john [options] [zip file] > [output file]
```

- `[options]` : Allows you to pass specific checksum options to `zip2john`; this shouldn't often be necessary
- `[zip file]` : The path to the Zip file you wish to get the hash of
- `>` : This redirects the output from this command to another file
- `[output file]` : This is the file that will store the output

Example Usage: `zip2john zipfile.zip > zip_hash.txt`

Cracking

We're then able to take the file we output from `zip2john` in our example use case, `zip_hash.txt`, and, as we did with `unshadow`, feed it directly into John as we have made the input specifically for it.

```
john --wordlist=/usr/share/wordlists/rockyou.txt zip_hash.txt
```

Practical: Now, have a go at cracking a "secure" Zip file! The file is located in `~/John-the-Ripper-The-Basics/Task09/`.

Answer the questions below

- | What is the password for the secure.zip file? pass123

```

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9# zip2john secure_1605054835063.zip > zip_hash.txt
ver 1.0 efh 5455 efh 7875 secure_1605054835063.zip/zippy/flag.txt PKZIP
Encr: 2b chk, TS_chk, cmplen=38, decmplen=26, crc=849AB5A6 ts=B689 cs=b689 type=0
Note: It is normal for some outputs to be very large
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9# ls
secure_1605054835063.zip zip_hash.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9# cat zip_hash.txt
secure_1605054835063.zip/zippy/flag.txt:$pkzip$1*2*2*0*26*1a*849ab5a6*0
*48*0*26*b689*964fa5a31f8cefe8e6b3456b578d66a08489def78128450ccf07c28df
a6c197fd148f696e3a2*$/$pkzip$:zippy/flag.txt:secure_1605054835063.zip::s
ecure_1605054835063.zip
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9# john --wordlist=/usr
/share/wordlists/rockyou.txt zip_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (PKZIP [32/64])
Will run 2 OpenMP threads
Note: Passwords longer than 21 [worst case UTF-8] to 63 [ASCII] rejecte
d
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for st
atus
pass123 (secure_1605054835063.zip/zippy/flag.txt)
1g 0:00:00:00 DONE (2026-06-24 08:51) 33.33g/s 273066p/s 273066c/s 2730
66C/s newzealand..total90
Use the "--show" option to display all of the cracked passwords reliabl
y
Session completed

```

What is the contents of the flag inside the zip file? THM{w3ll_d0n3_h4sh_r0y4l}

```

secure_1605054835063.zip zip_hash.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9# cat zip_hash.txt
secure_1605054835063.zip/zippy/flag.txt:$pkzip$1*2*2*0*26*1a*849ab5a6*0
*48*0*26*b689*964fa5a31f8cefe8e6b3456b578d66a08489def78128450ccf07c28df
a6c197fd148f696e3a2*$/$pkzip$:zippy/flag.txt:secure_1605054835063.zip::s
ecure_1605054835063.zip
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9# unzip secure_1605054
835063.zip
Archive: secure_1605054835063.zip
[secure_1605054835063.zip] zippy/flag.txt password:
extracting: zippy/flag.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9# ls
secure_1605054835063.zip zip_hash.txt zippy
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9# cd zippy
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9/zippy# ls
flag.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9/zippy# cat flag.txt
THM{w3ll_d0n3_h4sh_r0y4l}
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task9/zippy#

```

Task 10 - Cracking Password-Protected RAR Archives

Cracking a Password-Protected RAR Archive: We can use a similar process to the one we used in the last task to obtain the password for RAR archives. If you aren't familiar, RAR archives are compressed files created by the WinRAR archive manager. Like Zip files, they compress folders and files.

Rar2John: Almost identical to the `zip2john` tool, we will use the `rar2john` tool to convert the RAR file into a hash format that John can understand. The basic syntax is as follows:

```
rar2john [rar file] > [output file]
```

- `rar2john` : Invokes the `rar2john` tool
- `[rar file]` : The path to the RAR file you wish to get the hash of
- `>` : This redirects the output of this command to another file
- `[output file]` : This is the file that will store the output from the command

Example Usage `/opt/john/rar2john rarfile.rar > rar_hash.txt`

Cracking Once again, we can take the file we output from `rar2john` in our example use case, `rar_hash.txt`, and feed it directly into John as we did with `zip2john`.

```
john --wordlist=/usr/share/wordlists/rockyou.txt rar_hash.txt
```

Practical: Now, have a go at cracking a "secure" RAR file! The file is located in `~/John-the-Ripper-The-Basics/Task10/`.

Answer the questions below

What is the password for the secure.rar file? password

```
root@ip-10-113-98-242:~/Rooms/JohnTheRipper# cd task10
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# ls
secure_1605054844670.rar
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# rar2john secure_160
5054844670.rar > rar_hash.txt
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# ls
rar_hash.txt  secure_1605054844670.rar
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# john --wordlist=/us
r/share/wordlists/rockyou.txt rar_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (RAR5 [PBKDF2-SHA256 256/256 AVX2 8x])
Cost 1 (iteration count) is 32768 for all loaded hashes
Will run 2 OpenMP threads
Note: Passwords longer than 10 [worst case UTF-8] to 32 [ASCII] rejecte
d
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for st
atus
password      (secure_1605054844670.rar)
1g 0:00:00:00 DONE (2026-06-24 08:58) 1.299g/s 83.12p/s 83.12c/s 83.12C
/s 123456..charlie
Use the "--show" option to display all of the cracked passwords reliabl
y
Session completed
```

What are the contents of the flag inside the rar file? THM{r4r_4rch1ve5_th15_t1m3}

```
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# ls
rar_hash.txt  secure_1605054844670.rar
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# unrar
Command 'unrar' not found, but can be installed with:
apt install unrar-free # version 1:0.1.3-1, or
apt install unrar      # version 1:7.0.5-1
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# apt install unrar
Reading package lists... Done
Building dependency tree... Done
Reading state information... Done
The following NEW packages will be installed:
  unrar
0 upgraded, 1 newly installed, 0 to remove and 5 not upgraded.
Need to get 178 kB of archives.
After this operation, 407 kB of additional disk space will be used.
```

```

root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# ls
rar_hash.txt  secure_1605054844670.rar
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# unrar e rar_hash.tx
t
UNRAR 7.00 freeware      Copyright (c) 1993-2024 Alexander Roshal

rar_hash.txt is not RAR archive
No files to extract
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# unrar e secure_1605
054844670.rar
UNRAR 7.00 freeware      Copyright (c) 1993-2024 Alexander Roshal

Enter password (will not be echoed) for secure_1605054844670.rar:

Extracting from secure_1605054844670.rar

Extracting flag.txt
  OK
All OK
root@ip-10-113-98-242:~/Rooms/JohnTheRipper/task10# cat flag.txt
THM{r4r_4rch1ve5_th15_t1m3}

```

Task 11 - Cracking SSH Keys with John

Cracking SSH Key Passwords: Frequently in CTF challenges—using John to crack the SSH private key password of `id_rsa` files. Unless configured otherwise, you authenticate your SSH login using a password. However, you can configure key-based authentication, which lets you use your private key, `id_rsa`, as an authentication key to log in to a remote machine over SSH. However, doing so will often require a password to access the private key; here, we will be using John to crack this password to allow authentication over SSH using the key.

SSH2John: Who could have guessed it, another conversion tool? Well, that's what working with John is all about. As the name suggests, `ssh2john` converts the `id_rsa` private key, which is used to log in to the SSH session, into a hash format that John can work with. Jokes aside, it's another beautiful example of John's versatility. The syntax is about what you'd expect. Note that if you don't have `ssh2john` installed, you can use `ssh2john.py`, located in the `/opt/john/ssh2john.py`. If you're doing this on the AttackBox, replace the `ssh2john` command with `python3 /opt/john/ssh2john.py` or on Kali, `python /usr/share/john/ssh2john.py`.

```
ssh2john [id_rsa private key file] > [output file]
```

- `ssh2john` : Invokes the `ssh2john` tool
- `[id_rsa private key file]` : The path to the `id_rsa` file you wish to get the hash of
- `>` : This is the output director. We're using it to redirect the output from this command to another file.
- `[output file]` : This is the file that will store the output from

Example Usage: `/opt/john/ssh2john.py id_rsa > id_rsa_hash.txt`

Cracking: For the final time, we're feeding the file we output from `ssh2john`, which in our example use case is called `id_rsa_hash.txt` and, as we did with `rar2john`, we can use this seamlessly with John:

```
john --wordlist=/usr/share/wordlists/rockyou.txt id_rsa_hash.txt
```

Practical: Now, I'd like you to crack the hash of the `id_rsa` file relevant to this task! The file is located in `~/John-the-Ripper-The-Basics/Task11/`.

Answer the questions below

What is the SSH private key password? mango

```

root@ip-10-114-108-85:~/Rooms/JohnTheRipper/task11# john --wordlist=/usr/share/w
ordlists/rockyou.txt id_rsa_hash.txt
Using default input encoding: UTF-8
Loaded 1 password hash (SSH, SSH private key [MD5/bcrypt-pbkdf/[3]DES/AES 32/64]
)
Cost 1 (KDF/cipher [0:MD5/AES 1:MD5/[3]DES 2:bcrypt-pbkdf/AES]) is 0 for all loa
ded hashes
Cost 2 (iteration count) is 1 for all loaded hashes
Will run 2 OpenMP threads
Press 'q' or Ctrl-C to abort, 'h' for help, almost any other key for status
mango (id_rsa_1605800988509.id_rsa)
1g 0:00:00:00 DONE (2026-06-24 10:43) 9.091g/s 38981p/s 38981c/s 38981C/s praise
..mango
Use the "--show" option to display all of the cracked passwords reliably
Session completed

```